

Diseño ImgAnalysis pipeline (SCA 20240625)

ImgAnalysis es un IP que extrae distintas características del stream de video procedente del pipeline ImgConditioning. No se trata por tanto de un IP genético, ya que sólo acepta un stream de entrada en formato YUV y 2pixels por muestra (pps).

Características: Histograma, NumPix, SumValues.

- Interfaz de entrada: AXIStream YUV (2 pixels/clock)
- Interfaz de configuración: AXI_LITE (rows, cols)
- Intefaz salida de datos: AXI_LITE (histo[256], npix, sumval)

Limitaciones: el número de columnas de la imagen debe ser divisible entre 2

Referencias:

https://github.com/JoshuaEbenezer/vivado_hls

<http://www.aiotlab.org/teaching/fpga/Prefix-Sum-and-Histogram.pdf>

pxs_analysis.hcc

<http://www.aiotlab.org/teaching/fpga.html>

<https://github.com/KastnerRG/pp4fpgas>

Video IP: AXI Feature Adoption (UG1037): definición del protocolo de video de Xilinx para AXI-Stream

<https://docs.amd.com/v/u/en-US/ug1037-vivado-axi-reference-guide>

Table 4-6: AXI4-Stream Video Protocol Input (Slave) Interface Signals

Function	Width	Direction	AXI4-Stream Signal Name	Video Specific Name
Video Data	Any number of bytes	IN	s_axis_video_tdata	DATA
Valid	1	IN	s_axis_video_tvalid	VALID
Ready	1	OUT	s_axis_video_tready	READY
Start Of Frame	1	IN	s_axis_video_tuser	SOF
End Of Line	1	IN	s_axis_video_tlast	EOL

1. D
esa
rrol
lo y

simulación

- Interfaz

```
void imganalysis(yuv_stream& stream_in_48, uint32_t histo[NBINS], uint32_t*npix, uint32_t*sumval, uint32_t rows, uint32_t cols) {
```

```
  // #pragma HLS INTERFACE s_axilite port=return bundle=control
```

```
  #pragma HLS INTERFACE ap_ctrl_none port=return
```

```
  #pragma HLS INTERFACE s_axilite port=rows bundle=histo_data
```

```
  #pragma HLS INTERFACE s_axilite port=cols bundle=histo_data
```

```
  #pragma HLS INTERFACE s_axilite port=histo bundle=histo_data
```

```
  #pragma HLS INTERFACE s_axilite port=npix bundle=histo_data
```

```
  #pragma HLS INTERFACE s_axilite port=sumval bundle=histo_data
```

```
  #pragma HLS INTERFACE axis depth=24 port=stream_in_48 register
```

- Simulación C correcta con imagen 640x480

- Resultado de la síntesis:

Performance & Resource Estimates														
Modules & Loops	Issue Type	Violation Type	Distance	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT
imganalysis				-	307732	1.016E6	-	307733	-	no	6	3	1158	1422
imganalysis_Pipeline_Init				-	258	851.000	-	258	-	no	0	0	11	54
Init				-	256	845.000	1	1	256	yes	-	-	-	-
imganalysis_Pipeline_histo_loop				-	307203	1.014E6	-	307203	-	no	0	0	199	402
histo_loop				-	307201	1.014E6	3	1	307200	yes	-	-	-	-
imganalysis_Pipeline_hist_Reduce_Clear				-	260	858.000	-	260	-	no	0	0	185	142
hist_Reduce_Clear				-	258	851.000	4	1	256	yes	-	-	-	-

2. Espacio de Registros

Dirección del registro	R/W	Descripción
0x0010	R	Npix: número de pixeles en la imagen
0x0014	R	SumVal: suma de los valores de todos los píxels
0x0030	W	Rows: número de filas de la imagen
0x0038	W	Cols: número de columnas de la imagen
0x0400 .. 0x07ff	R	Histo[i]: valores del histograma i=0 ..255

3. Verificación en placa prototipado

Se ha integrado en un pipeline de captura de imagen MIPI genérico e implementado sobre la placa Kria kv260. Para la parte sw se ha utiliza el driver por defecto de las librerías PYNQ y un Jupiter Notebook para la configuración y verificación de los resultados.

Importante:

- el IP debe instertarse a continuación del módulo *AXI4-Stream Subset Converter*
- para su integración debe utilizarse el IP *AXI4-Stream Broadcaster* que divide el stream original en 2 streams idénticos: uno continúa con el flujo original del pipeline, el otro se emplea para realizar el análisis con el IP *ImgAnalysis*

